01_DATA_ARCHITECTURE

SUOP ERP v1.0 — Enterprise Data Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval

Field	Value
Date	2026-07-11
Supersedes	All prior database decisions
Approval Required	Project Owner

---

1. Purpose

This document defines how data is structured, stored, accessed, protected, retired, and recovered in SUOP ERP v1.0. Every Prisma model, every migration, every query, and every archiving decision must conform to this document.

Hard rule: No Prisma schema change is permitted unless it conforms to the conventions defined here. Schema Review Reports must reference this document section by section.

---

2. Database Platform

2.1 Primary Platform

- **Production:** PostgreSQL 16+ (managed — Supabase, RDS, or Aurora)
- **Staging:** PostgreSQL 16+ (managed, smaller instance)
- **Development:** PostgreSQL 16+ in Docker (via `docker-compose`)
- **Test:** PostgreSQL 16+ via Testcontainers (isolated per test run)

Justification: PostgreSQL is the only open-source database that meets all enterprise ERP requirements: ACID, row-level security, JSONB, full-text search, partitioning, logical replication, point-in-time recovery, extensions (pgcrypto, pg_trgm, uuid-ossp).

Forbidden: SQLite (no row-level security, no concurrent write performance), MySQL (weaker transaction isolation, no JSONB), MongoDB (not relational — ERP requires relational integrity).

2.2 Connection Pooling

- **Production:** PgBouncer or Supabase's built-in Supavisor (transaction-mode pooling, pool size 20-50)
- **Application:** Prisma connection pool — `DATABASE_POOL_SIZE=10` per instance, scaled horizontally
- **Max connections:** 100 (Postgres default) → 80 application + 20 reserved for admin/migration
- **Connection lifecycle:** Idle timeout 30s; max lifetime 5min; Prisma logs warnings on slow acquisitions (>1s)

2.3 Character Set & Collation

- **Encoding:** `UTF8` (mandatory — supports all Indian languages + emoji)
- **Collation:** `en_US.utf8` for default; per-column collation for case-insensitive search
- **Timezone:** DB stored in `UTC`; application converts to tenant/user timezone for display

---

3. Database Naming Conventions

3.1 Tables

- **Format:** `snake_case`, plural
- **Examples:** `purchase_orders`, `goods_receipts`, `stock_ledger_entries`, `audit_logs`
- **Join tables:** `singular_singular` (e.g., `user_role`, `role_permission`)
- **Prefix conventions:**
  - No prefix for business tables (`purchase_orders`)
  - `_meta_` prefix for framework tables
  - `vw_` prefix for views
  - `fn_` prefix for functions

3.2 Columns

- **Format:** `snake_case`
- **Primary key:** Always `id` (UUID v7)
- **Foreign keys:** `<entity>_id` (e.g., `supplier_id`, `purchase_order_id`)
- **Timestamps:** `created_at`, `updated_at`, `deleted_at` (always)
- **Actor references:** `created_by`, `updated_by`, `deleted_by`
- **Status:** `status` (string, never boolean — must support state machine)
- **Booleans:** `is_<adjective>` (e.g., `is_active`, `is_immutable`)
- **Amounts:** Always `Decimal(14, 2)` for money, `Decimal(14, 3)` for quantities
- **Avoid abbreviations:** `quantity` not `qty`, `purchase_order_id` not `po_id` in schema

3.3 Indexes

- **Format:** `idx_<table>_<columns>` (e.g., `idx_purchase_orders_supplier_id_status`)
- **Unique indexes:** `uq_<table>_<columns>`
- **Composite indexes:** Order columns by selectivity (most selective first)
- **Partial indexes:** `WHERE deleted_at IS NULL` for soft-deleted tables
- **Foreign key indexes:** Every FK gets an index automatically
- **GIN indexes:** For JSONB columns queried with `@>` operator

3.4 Constraints

- **Check constraints:** Named `ck_<table>_<rule>`
- **Unique constraints:** Named `uq_<table>_<columns>`
- **Foreign key constraints:** Named `fk_<from_table>_<to_table>`

3.5 Reserved Columns (every business table)

```
id              UUID          PRIMARY KEY DEFAULT
        uuid_generate_v7()
tenant_id       UUID          NOT NULL
version         INTEGER       NOT NULL DEFAULT 0
created_at      TIMESTAMPTZ   NOT NULL DEFAULT NOW()
created_by      UUID          NULL
updated_at      TIMESTAMPTZ   NOT NULL DEFAULT NOW()
updated_by      UUID          NULL
deleted_at      TIMESTAMPTZ   NULL
deleted_by      UUID          NULL
```

---

4. Primary Key Strategy

4.1 Type: UUID v7

- **Format:** 128-bit UUID, time-ordered (RFC 9562)
- **Generation:** PostgreSQL `uuid_generate_v7()` OR application-side generation via `uuidv7` npm package
- **Storage:** `UUID` type (16 bytes — half the size of `TEXT`)

4.2 Why UUID v7 (not v4, not BIGSERIAL)

Property	UUID v4	BIGSERIAL	UUID v7
Globally unique	Yes	No (centralized)	Yes
Time-sortable	No	Yes	Yes
Insert-perf (random PK)	No (B-tree fragmentation)	Yes	Yes (sequential-ish)
Predictable count	No	No (leaks scale)	No

Property	UUID v4	BIGSERIAL	UUID v7
No coord needed	Yes	No	Yes
URL-safe (no int overflow)	Yes	No	Yes

Critical: UUID v7 is time-ordered — inserts are append-mostly at the B-tree leaf, avoiding fragmentation.

4.3 Application-Side vs DB-Side Generation

- **Default:** DB-side (`DEFAULT uuid_generate_v7()`) — single source of truth
- **Exception:** Application-side when the application needs the ID before insert. Use `uuidv7()` npm package.

4.4 No Composite Primary Keys

- Every table has a single `id` UUID PK
- Composite uniqueness enforced via unique constraints
- Rationale: Simplifies ORM, simplifies foreign keys, simplifies audit log references

---

5. Tenant Strategy

5.1 Multi-Tenancy Model

Chosen model: Shared database, shared schema (with `tenant_id` discriminator)

Model	Pros	Cons	Verdict
Database per tenant	Strong isolation	Expensive, hard analytics	No

Model	Pros	Cons	Verdict
Schema per tenant	Some isolation	Migration pain	No
Shared DB, shared schema, `tenant_id`	Cheap, easy analytics	Weaker isolation, app-enforced	**Yes**

5.2 Tenant Isolation Enforcement (3 layers)

Layer 1 — Application (Prisma extension): - AsyncLocalStorage carries `tenantId` from the request - Prisma extension injects `WHERE tenant_id = $currentTenant` into every query - Cross-tenant queries require explicit `withTenant()` + `system:tenant:cross` permission

Layer 2 — Database (PostgreSQL Row-Level Security):

```
ALTER TABLE purchase_orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON purchase_orders
  USING (tenant_id =
         current_setting('app.current_tenant_id')::uuid);
```

- Application sets `SET app.current_tenant_id = '...'` on every connection checkout
- Even if app has a bug, DB rejects cross-tenant reads/writes
- Superuser bypasses RLS (for migrations only — application role never superuser)

Layer 3 — Network/API: - JWT contains `tenantId` claim - TenantMiddleware validates token tenant matches request path tenant - Cross-tenant admin operations require permission AND audit log at `CRITICAL` severity

5.3 Tenant Context

```
interface TenantContext {
  tenantId: TenantId
  tenantName: string
  tenantTimeZone: string       // 'Asia/Kolkata'
  tenantCurrency: string       // 'INR'
  tenantLocale: string         // 'en-IN'
  tenantFeatures: string[]
}
```

5.4 Tenant Provisioning

- New tenant creation is a system-level operation
- Creates `tenants` row + default admin user + default roles + permissions
- Audit logged at `CRITICAL` severity; alerts security team

---

6. Index Strategy

6.1 Indexing Principles

1. **Every foreign key gets an index.** Postgres does not auto-create FK indexes.
2. **Every query path gets an index.** EXPLAIN ANALYZE must show index usage. No seq scans on tables >10k rows.
3. **Partial indexes for soft-deleted tables.** `WHERE deleted_at IS NULL`.
4. **Composite indexes for filter+sort.**
5. **No indexes on low-cardinality columns alone.**
6. **Covering indexes for hot queries.** Use `INCLUDE` for index-only scans.

6.2 Standard Index Patterns

```
-- Every business table
CREATE INDEX idx_<table>_tenant_id ON <table> (tenant_id)
        WHERE deleted_at IS NULL;
CREATE INDEX idx_<table>_created_at ON <table> (created_at
        DESC);

-- Lookup by business code
CREATE UNIQUE INDEX uq_<table>_<code>_tenant ON <table>
        (tenant_id, <code>) WHERE deleted_at IS NULL;

-- Foreign keys
CREATE INDEX idx_<table>_<fk_column> ON <table>
        (<fk_column>);

-- Status + tenant (common dashboard filter)
CREATE INDEX idx_<table>_tenant_status ON <table>
        (tenant_id, status) WHERE deleted_at IS NULL;
```

6.3 Index Review Process

- Every Prisma model must declare indexes in the schema
- CI verifies every FK has an index
- Unused indexes (30 days, `idx_scan = 0`) removed
- Index bloat monitored; rebuilt with `REINDEX CONCURRENTLY`

6.4 Full-Text Search

- Postgres `tsvector` + GIN index for FTS
- `pg_trgm` extension for fuzzy matching (Indian language support)
- No Elasticsearch in Phase 0 — Postgres FTS sufficient for ERP-scale

---

7. Partition Strategy

7.1 When to Partition

Partition tables expected to exceed **10 million rows** or **50GB**.

7.2 Partition Candidates (Phase 0)

Table	Expected Growth	Partition Strategy
`audit_logs`	~1M/month	Range partition by `created_at` (monthly)
`stock_ledger_entries`	~500K/month	Range partition by `posted_at` (monthly)
`event_outbox`	~2M/month	Range partition by `created_at` (weekly) + retention
`notification_outbox`	~500K/month	Range partition by `created_at` (weekly) + retention

7.3 Partition Type

- **Range partitioning** on timestamp columns (most common)
- **List partitioning** only for tenant-isolated tables >100GB (rare)

7.4 Partition Maintenance

- Scheduled job creates next month's partitions 1 week in advance
- Old partitions dropped via `DROP PARTITION` after retention (instant operation)
- Tool: `pg_partman` extension

---

8. Archiving Strategy

8.1 Data Categories

Category	Examples	Hot	Warm	Cold
Transactional	purchase_orders, grns, ledger	0-2 years	2-7 years	7+ years
Audit	audit_logs	0-1 year	1-3 years	3-7+ years
Operational	notifications, outbox	0-30 days	—	— (purged)
Master	products, suppliers, users	Always	—	—
Reference	currencies, countries	Always	—	—
Documents	coa_documents, evidence	0-3 years	3-7 years	7+ years (Glacier)

8.2 Archive Process

1. **Identify:** Daily job finds records past hot-window age → `archive_jobs` table
2. **Export:** Export to compressed JSONL in S3 (`s3://suop-archive/<tenant>/<table>/<yyyy-mm>.jsonl.gz`)
3. **Verify:** Re-import to temp table, compare counts + checksums
4. **Purge:** Delete from primary in batches of 1000; audit log entry written
5. **Cold Storage:** 90 days in S3 Standard → Glacier; 7 years → delete (or extend per legal hold)

8.3 Restore from Archive

- Admin API: `POST /api/_internal/archive/restore`

- Reads JSONL from S3, inserts back, resets `archived_at`
- Audited at `CRITICAL` severity
- Slow (intentional) — discourages casual use

---

9. Backup Strategy

9.1 Backup Layers

Layer 1 — Managed automated backups (Supabase/RDS): - Daily full backup (2 AM UTC) - PITR via WAL streaming — retention 14 days - Cross-region replica for DR

Layer 2 — Logical backups (pg_dump): - Daily `pg_dump` to S3 (3 AM UTC) - Compressed, encrypted (S3 SSE-KMS) - Retention: 30 days daily, 12 months weekly, 7 years monthly (compliance) - Verified by daily restore-test job

Layer 3 — File storage backups: - S3 versioning on all buckets - S3 cross-region replication to DR region - MinIO (dev) backed up via `mc mirror` to local NAS weekly

9.2 Backup Encryption

- All backups encrypted at rest (S3 SSE-KMS)
- Keys rotated annually
- Decryption keys in secrets manager (different region from backups)

9.3 Backup Integrity

- **Daily:** Restore-test to isolated DB, `pg_checksums`, sample queries
- **Weekly:** Full restore to staging, full E2E suite
- **Monthly:** DR failover drill — promote replica, smoke tests, failback

9.4 Backup Retention Policy

Backup Type	Retention	Storage
PITR WAL	14 days	Supabase managed
Daily logical	30 days	S3 Standard
Weekly logical	12 months	S3 Standard-IA
Monthly logical	7 years	S3 Glacier
File storage	7 years	S3 Glacier

---

10. Restore Strategy

10.1 Restore Scenarios

Scenario	Method	RTO Target
Accidental row delete (within 14 days)	PITR → extract row → insert	1 hour
Accidental table drop (within 14 days)	PITR → restore table	2 hours
Logical corruption (bad migration)	Restore nightly logical backup	4 hours
Region failure	Promote cross-region replica	30 minutes
Catastrophic (no primary, no replica)	Restore monthly Glacier backup	24 hours

10.2 Restore Procedure (Logical Backup)

1. Provision new Postgres instance (same version)

2. Download backup from S3, verify checksum
3. `pg_restore --clean --if-exists --jobs=4 <backup>`
4. Run `pg_checksums` for integrity
5. Run smoke tests
6. Switch application `DATABASE_URL`
7. Verify application functionality
8. Decommission old instance after 7-day observation

10.3 Restore Procedure (PITR)

1. Use Supabase/RDS console to restore to timestamp T (within 14 days)
2. New instance provisioned with data up to T
3. Application switched to new instance
4. Lost data (T → current) re-entered or replayed from event outbox

10.4 Restore Testing

- Monthly restore drill (full restore to staging)
- Quarterly DR failover drill (promote replica)
- Annual full disaster simulation (region loss, restore from Glacier)

---

11. Read/Write Patterns

11.1 Read Patterns

Pattern A — Direct lookup by ID: PK index, instant **Pattern B — Filtered list with pagination:** Composite index `(tenant_id, created_at DESC)`; cursor pagination preferred **Pattern C — Aggregation (dashboard):** Partial index; result cached in Redis 5 min **Pattern D — Join (master + transactional):** FK index critical **Pattern E — Full-text search:** `pg_trgm` GIN index for fuzzy match

11.2 Write Patterns

Pattern A — Single insert: ID pre-generated by app for child inserts in same transaction **Pattern B — Update with optimistic locking:**

```
UPDATE purchase_orders
SET status = $1, updated_at = NOW(), updated_by = $2,
        version = version + 1
WHERE id = $3 AND tenant_id = $4 AND version = $5 AND
        deleted_at IS NULL;
-- If 0 rows: either 404 or 409 (version mismatch)
```

Pattern C — Transactional multi-table: All-or-nothing; uses transaction helper + outbox pattern **Pattern D — Bulk insert:** Batch size 500; Prisma `createMany`; transactional **Pattern E — Soft delete:** Sets `deleted_at`, `deleted_by`, increments version

11.3 Forbidden Patterns

- ❌ `SELECT *` in production (explicit column list)
- ❌ N+1 queries (use Prisma `include` or DataLoader)
- ❌ `OFFSET` for deep pagination (cursor-based)
- ❌ `DELETE` without `WHERE` (always soft delete)
- ❌ Cross-tenant queries without explicit `withTenant()` + permission check

---

12. Data Lifecycle

12.1 Lifecycle Stages

```
Create → Use → Update → Soft Delete → Archive → Cold Storage
→ Purge
```

12.2 Lifecycle Rules per Category

Master Data (products, suppliers, users): - Never archived while referenced by transactional data - Soft delete blocks new references; existing preserved - Hard purge only after 7 years of zero references (compliance audit required)

Transactional Data (POs, GRNs, stock ledger): - Hot: 2 years online - Warm: years 2-7 archived to S3 - Cold: years 7+ to Glacier - Purge: never (legal hold for audits)

Audit Logs: Hot 1y → Warm 1-3y → Cold 3-7+y → Purge: never without legal sign-off **Operational Data (notifications, outbox):** Hot 30d → Purge **File Storage:** Hot 3y → Warm 3-7y → Cold 7+y → Purge: never without legal sign-off

12.3 Lifecycle Automation

- Daily: `archive-sweeper`, `outbox-purger`
- Weekly: `glacier-transition`
- Monthly: `archive-compressor`
- All jobs audit-logged; failures alert ops

---

13. Historical Data Policy

13.1 What Counts as “Historical”

- Closed POs, GRNs, NCRs, CAPAs older than 2 years
- Stock ledger entries older than 2 years
- Audit logs older than 1 year

13.2 Access to Historical Data

- Read-only — no edits, no deletes (beyond archive purge)
- Requires `history:read` permission (separate from regular read)
- UI clearly labels with “Historical” badge
- Reports span historical + current via `UNION ALL`

13.3 Historical Snapshots

For trend dashboards: - Pre-computed nightly into `snapshot_<metric>` tables - Immutable once written - Past months never recomputed

13.4 Data Retention Compliance

Regulation	Data Type	Retention
FSSAI	Manufacturing records, COAs	2 years post-expiry
GST	Invoices, GRNs	6 years from filing
Companies Act	Financial records	8 years
Internal	Audit logs	7 years
Internal	Customer complaints	5 years

Configurable per tenant; default to most-strict if not set.

---

14. Master Data Policy

14.1 What is Master Data

Products, Suppliers, Customers, Warehouses, Plants, Users, Roles, UOMs, Currencies, Tax codes — entities that change rarely and are referenced by many transactional records.

14.2 Master Data Rules

1. **Single source of truth per tenant** — no duplicates; dedup runs nightly
2. **Soft delete only** — never hard delete
3. **Versioning for changes** — material changes create new versions; old preserved with `valid_to`
4. **Approval workflow for creation** — configurable per master type
5. **External ID support** — multiple external IDs (GSTIN, TIN, customer-assigned)
6. **Reference integrity** — UUID FK enforced
7. **Cache strategy** — Redis TTL 5 min; invalidated on update

14.3 Master Data Quality

- Required fields at schema level (NOT NULL)
- Format validation at service level (GST regex, PAN regex)
- Duplicate detection — fuzzy match flagged for review
- Completeness score — % optional fields populated
- Staleness alerts — 12 months without update

---

15. Transaction Data Policy

15.1 What is Transaction Data

POs, GRNs, stock issues, transfers, stock ledger entries, NCRs, CAPAs, COAs, audit logs, event outbox.

15.2 Transaction Data Rules

1. **Immutable after closure** — corrections via reversal + new transaction (double-entry)
2. **Sequence numbers** — tenant-unique sequence (e.g., `PO-2026-000001`) via Postgres sequence
3. **Status workflow** — every transaction has state machine
4. **Reference integrity** — master record soft-delete blocked if transactions reference it
5. **Posting to ledger** — financial/stock transactions post in same DB transaction
6. **Reversal pattern** — counter-transaction with reference to original; both preserved

15.3 Sequence Number Generation

```
CREATE SEQUENCE po_sequence_<tenant_id> START 1;
```

- Format: `<prefix>-<year>-<6-digit-sequence>` (e.g., `PO-2026-000001`)

- Resets yearly (separate sequence per year per tenant)
- Generated inside the creating transaction (atomic)

---

16. Reference Data Policy

16.1 What is Reference Data

System-wide constants shared across tenants: countries, currencies, UOMs, tax codes, languages, timezones, product categories.

16.2 Reference Data Rules

1. **Stored in `reference_*` tables** — no `tenant_id` column
2. **Tenant overrides** — `tenant_reference_overrides` table
3. **Versioned** — changes create new versions; effective-dated
4. **Seeded via migrations** — updates via migration only (not API)
5. **Cache aggressively** — Redis TTL 1 hour; invalidated via event
6. **Read-only API** — `GET /api/v1/reference/<type>`; no mutations

16.3 Reference Data Updates

- Migration only (code-reviewed)
- Emergency updates (GST rate change) via admin API with `system:reference:update` permission
- Each update publishes `ReferenceDataUpdated` event

---

17. Schema Migration Discipline

17.1 Migration Workflow

1. Modify `prisma/schema.prisma`
2. `prisma migrate dev --name <descriptive>` creates migration SQL
3. Review generated SQL — no destructive changes
4. Run locally + test suite
5. PR includes migration file

6. CI runs migration against fresh DB + full test suite
7. Staging auto-deploys on merge
8. Production via CD pipeline with manual approval

17.2 Migration Rules

- **Never destructive in one step** — column drops require 2 migrations (deprecate, then drop after 1 release)
- **Never change column type in one step** — add new, migrate data, drop old
- **Never rename columns** — add new, dual-write, migrate reads, drop old
- **Never rely on application downtime** — all migrations forward-compatible
- **Always include rollback SQL** — paired with every migration; tested in CI

17.3 Migration Review Checklist

- [ ] No `DROP TABLE` without prior deprecation release
- [ ] No `DROP COLUMN` without prior deprecation release
- [ ] No `ALTER TYPE` requiring table rewrite
- [ ] No `SET NOT NULL` without default for existing rows
- [ ] New indexes created with `CONCURRENTLY`
- [ ] FK constraints added with `NOT VALID` then `VALIDATE`
- [ ] Migration tested against production-size data in staging

---

18. Database Access Discipline

18.1 Application Role

- Non-superuser role
- CRUD on business tables, SELECT on reference tables
- Cannot `TRUNCATE`, `DROP`, or `ALTER`
- `EXECUTE` on whitelisted stored functions

18.2 Migration Role

- Separate role for migrations
- `CREATE`, `ALTER`, `DROP` privileges
- CI/CD pipeline only; credentials in secrets manager

18.3 Read Replica Role

- Read-only for reporting/analytics
- Connects to read replica
- Dashboards use this role to offload primary

18.4 Forbidden

- ❌ Application uses superuser role
- ❌ Direct DB access from developer machines to production
- ❌ Long-running queries without `statement_timeout`
- ❌ Manual schema changes outside migrations

---

19. Database Observability

19.1 Metrics (Prometheus)

- Connections: active, idle, waiting
- QPS by type (SELECT, INSERT, UPDATE, DELETE)
- Query latency p50, p95, p99
- Transaction duration p50, p95, p99

- Lock waits (count + duration)
- Cache hit ratio
- Index usage
- Table bloat

19.2 Slow Query Log

- `log_min_duration_statement = 500ms`
- Top 10 slowest queries reviewed weekly
- Added to optimization backlog

19.3 Query Plan Analysis

- Weekly `pg_stat_statements` sampling for top 20 expensive queries
- EXPLAIN ANALYZE run automatically; plan stored
- Seq scans on tables >10k rows flagged

19.4 Lock Monitoring

- Alert if any lock wait exceeds 5s
- Alert on deadlocks
- Long-running transactions (>30s) alert ops

---

20. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-D1	Postgres hosting	Supabase Pro	AWS RDS, Aurora
Q-D2	UUID generation	DB-side `uuid_generate_v7()`	App-side
Q-D3	Partitioning tool	`pg_partman`	Custom cron
Q-D4		S3 Glacier	

#	Decision	Recommendation	Alternatives
	Archive storage		Azure Blob Cool, GCS Archive
Q-D5	Backup verification	Daily restore-test	Weekly
Q-D6	Reference data updates	Migration-only	Admin API for emergency
Q-D7	Read replica for dashboards	Yes (Supabase)	No (use primary)

Approval Block

Approved by: _____ Date: _____

End of Enterprise Data Architecture Document